CT-159 · DATA STRUCTURES ALGORITHM APPLICATION

Linked Lists

Lecture 2 — Singly, Doubly & Circular Linked Lists

Presenter: Dr. Muhammad Kamran

SECTION 01

Linked List Basics

A chain of nodes, each pointing to the next — a flexible alternative to arrays.

SECTION 01 · BASICS

01

What is a Linked List?

A linear structure made of nodes, each connected to the next by a pointer (or reference).

- Each node contains data and a pointer/reference to the next node.
- Nodes are not stored contiguously — they can live anywhere in memory.
- This makes insertion and removal at any position efficient, unlike arrays.

SECTION 01 · BASICS

02

Linked List vs Array

Linked List	Array
Memory: non-contiguous	**Memory:** contiguous
Allocation: one node at a time	**Allocation:** whole block at once
Insert/Delete: efficient	**Insert/Delete:** inefficient
Access: sequential	**Access:** random O(1)

SECTION 01 · BASICS

03

A Sequence of Links

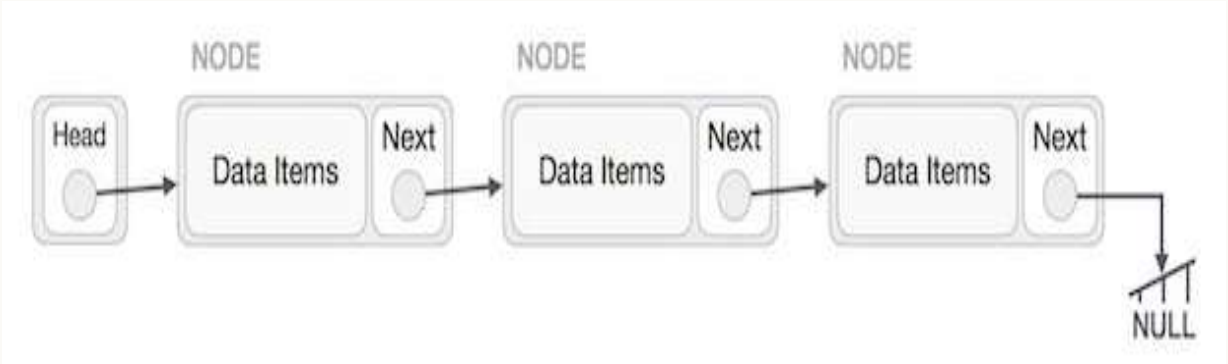
- A linked list is a sequence of data structures connected together via links.
- Each link contains items and a connection to another link.
- It is the second most-used data structure after the array.

SECTION 01 · BASICS

04

Linked List Representation

Head points to the first node; each node points to the next; the last points to NULL.



SECTION 01 · BASICS

05

Key Terms

Head	The entry point — a link to the first node of the list.
Data field	Each node holds one or more data values.
Next	A link in each node pointing to the following node.
NULL	The last node's next is NULL, marking the end.

SECTION 01 · BASICS

06

Three Types of Linked List

Singly Linked List	Navigation is forward only — each node points to the next.
Doubly Linked List	Navigation both ways — each node points to next and previous.
Circular Linked List	The last node links back to the first — no NULL end.

SECTION 02

Operations

Insertion, deletion, search — and why linked lists shine here.

Basic Operations

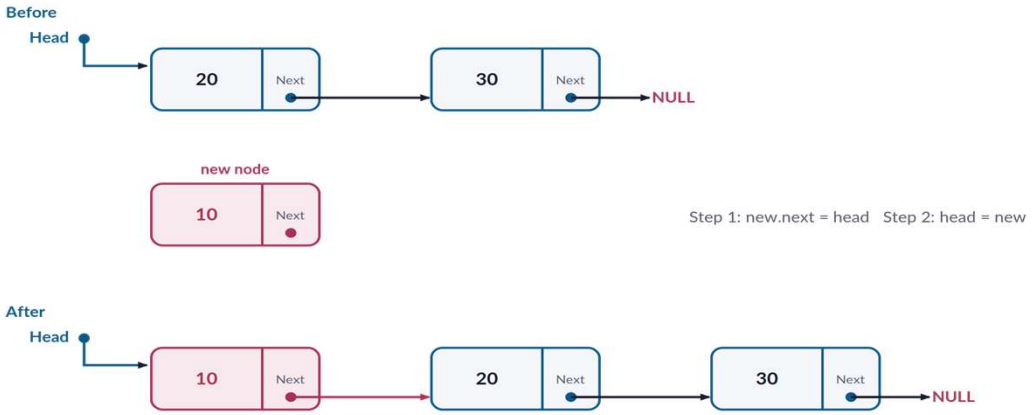
Insertion	Add an element (e.g. at the beginning of the list).
Deletion	Remove an element from the list.
Display	Traverse and show the complete list.
Search	Find an element using a given key.
Delete (by key)	Remove a specific element matching the given key.

SECTION 02 · OPERATIONS

08

Insertion at Head

Add a node at the front — the fast case: no walking needed.



SECTION 02 · OPERATIONS

09

Insertion at Head — Algorithm

Steps

1. Create a new node with the value
2. Set new.next = head
3. Set head = new
4. Done — O(1), constant time

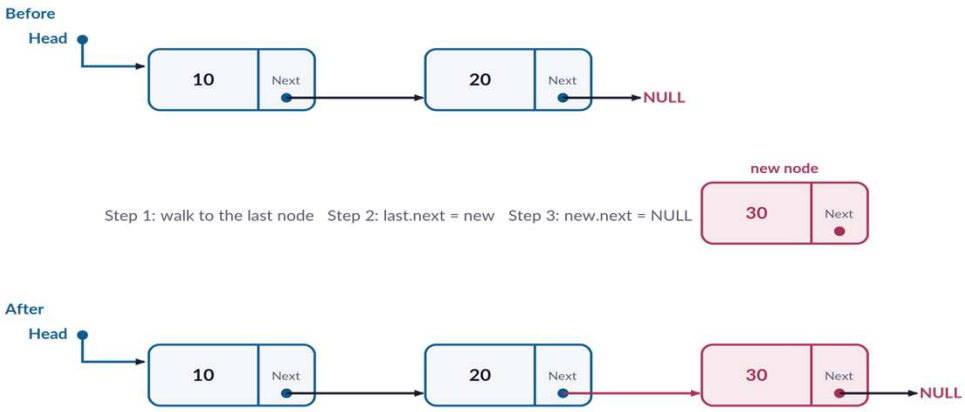
```
// pseudocode
newNode = Node(value)
newNode.next = head
head = newNode
```

SECTION 02 · OPERATIONS

10

Insertion at Tail

Add a node at the end — must first walk to the last node.



SECTION 02 · OPERATIONS

11

Insertion at Tail — Algorithm

Steps

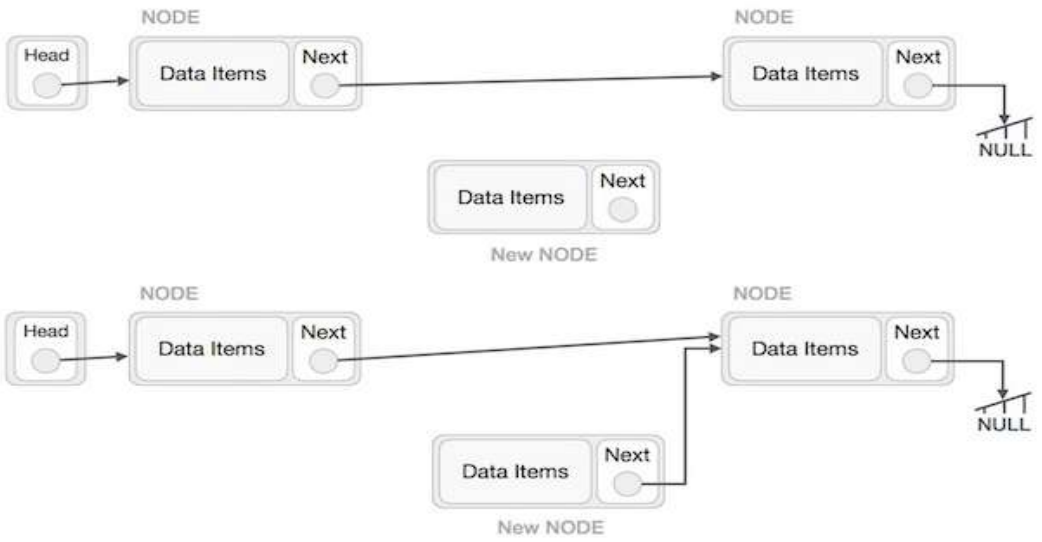
1. Create a new node; new.next = NULL
2. If list is empty: head = new, stop
3. Start temp at head
4. Walk while temp.next != NULL
5. Set temp.next = new — O(n)

```
// pseudocode
newNode = Node(value)
newNode.next = NULL
if head == NULL:
    head = newNode
    return
temp = head
while temp.next != NULL:
    temp = temp.next
temp.next = newNode
```

SECTION 02 · OPERATIONS

12

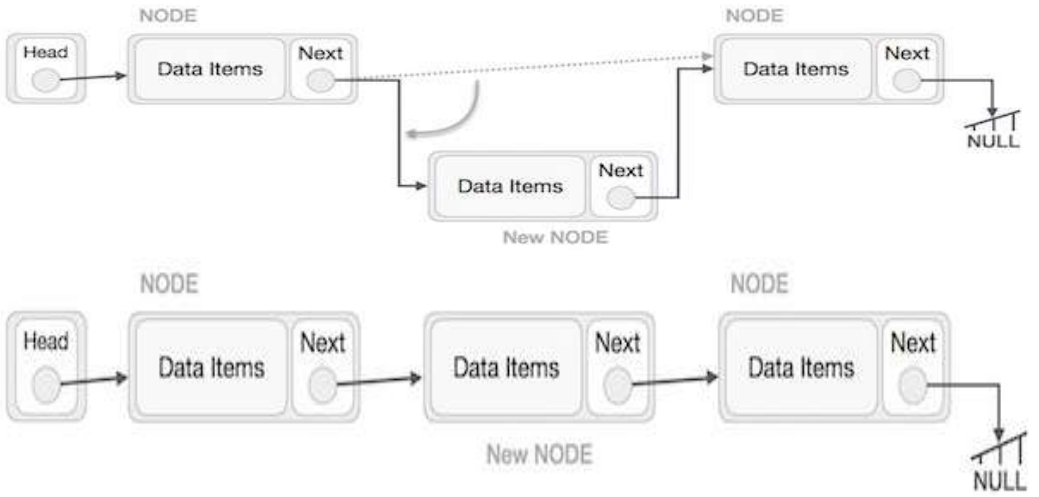
Insertion in the middle



SECTION 02 · OPERATIONS

13

Insertion in the middle



SECTION 02 · OPERATIONS

14

Insertion in Middle — Algorithm

Steps

1. Reach the node you want to insert after (curr)
2. Create the new node
3. new.next = curr.next
4. curr.next = new — order matters!

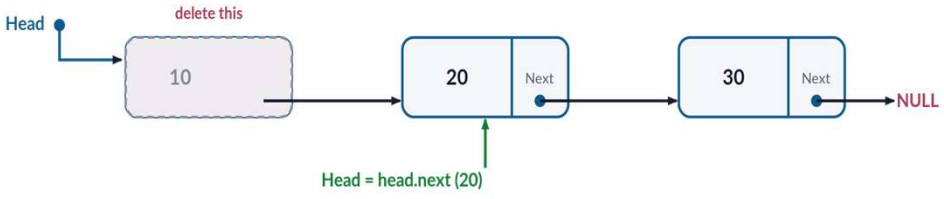
```
// pseudocode
newNode = Node(value)
newNode.next = curr.next
curr.next = newNode
// set new.next FIRST, or the
// rest of the list is lost
```

SECTION 02 · OPERATIONS

15

Deletion from Head

Remove the first node — the fast case.



Move Head to the second node, then free the old first node.

SECTION 02 · OPERATIONS

16

Deletion from Head — Algorithm

Steps

1. If list is empty: nothing to delete
2. temp = head
3. head = head.next
4. free(temp) — O(1)

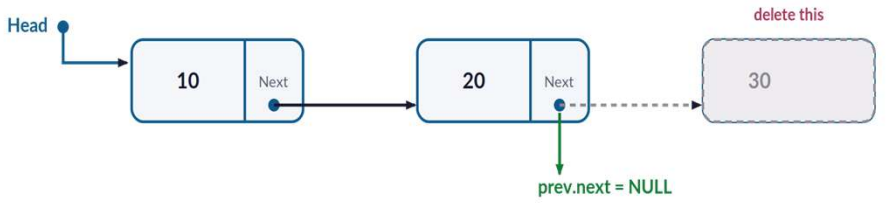
```
// pseudocode
if head == NULL:
    return
temp = head
head = head.next
free(temp)
```

SECTION 02 · OPERATIONS

17

Deletion from Tail

Remove the last node — walk to the second-last node first.



Walk to the second-last node, set its next to NULL, then free the last node.

SECTION 02 · OPERATIONS

18

Deletion from Tail — Algorithm

Steps

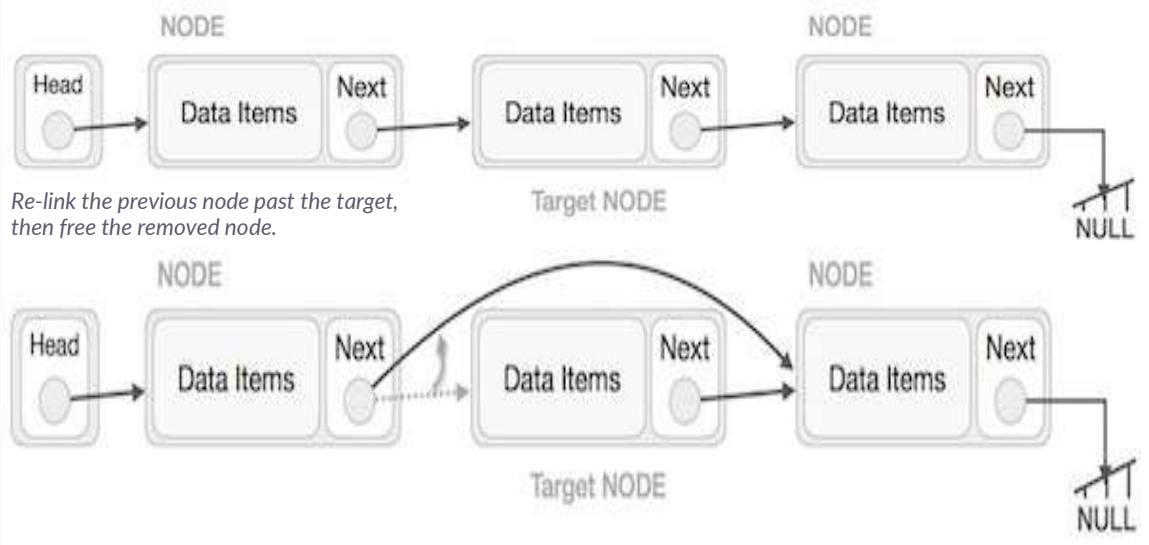
1. If empty or single node: handle directly
2. temp = head
3. Walk while temp.next.next != NULL
4. free(temp.next)
5. temp.next = NULL — O(n)

```
// pseudocode
if head == NULL: return
if head.next == NULL:
    free(head); head = NULL; return
temp = head
while temp.next.next != NULL:
    temp = temp.next
free(temp.next)
temp.next = NULL
```

SECTION 02 · OPERATIONS

19

Deletion from the middle

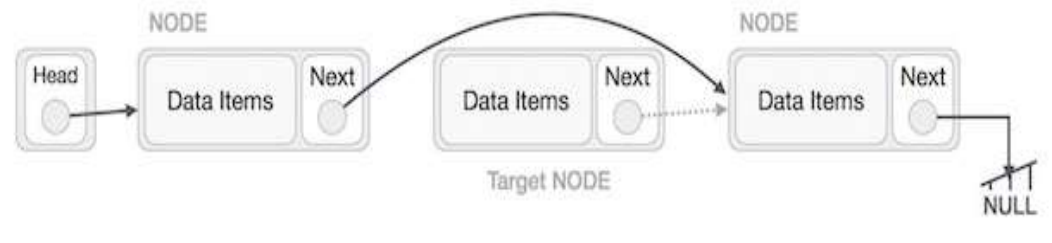


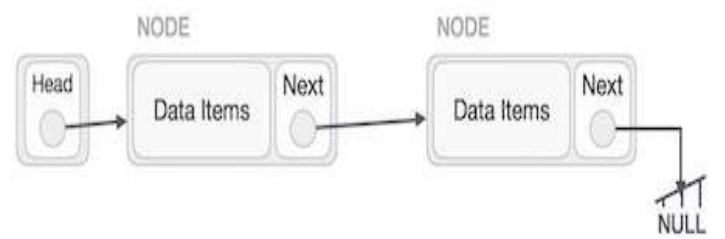
Re-link the previous node past the target, then free the removed node.

SECTION 02 · OPERATIONS

20

Deletion from the middle





SECTION 02 · OPERATIONS

21

Deletion from Middle — Algorithm

Steps

1. Find prev (the node before the target)
2. target = prev.next
3. prev.next = target.next
4. free(target)

```
// pseudocode

// prev points to the node
// before the one to delete
target = prev.next
prev.next = target.next
free(target)
```

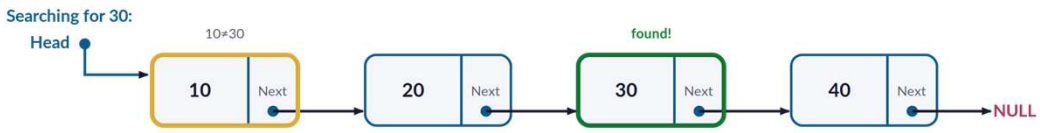
SECTION 02 · OPERATIONS

22

Searching a Linked List

No index means no binary search — we walk the list (linear).

Linked lists have no index, so we can only walk node by node (linear).



Start at Head; compare each node's data with the key; stop when it matches or you reach NULL.

Linear search on a list: O(n)

Binary search needs O(1) index access, so it does NOT work on a plain linked list.

SECTION 02 · OPERATIONS

23

Search — Linear vs Binary

Steps

1. Start temp at head
2. Compare temp.data with the key
3. If equal → found, return position
4. Else temp = temp.next
5. Stop at NULL → not found (O(n))

```
// pseudocode
temp = head
index = 0
while temp != NULL:
    if temp.data == key:
        return index      // found
    temp = temp.next
    index = index + 1
return -1                 // not found

// Binary search needs O(1)
// random access -> arrays
// only, NOT linked lists.
```

SECTION 02 · OPERATIONS

24

Displaying the List

Traverse from head to NULL, printing each node.



Start at Head; print each node's data; follow next until you reach NULL.

Output: 10 → 20 → 30 → NULL

SECTION 02 · OPERATIONS

25

Display (Traverse) — Algorithm

Steps

1. Start temp at head
2. While temp != NULL:
3. print temp.data
4. temp = temp.next
5. Prints every node in order — O(n)

```
// pseudocode
temp = head
while temp != NULL:
    print(temp.data)
    temp = temp.next
// visits each node once,
// front to back
```

SECTION 03

Variations

Doubly and circular linked lists — and how to reverse a list.

Doubly Linked List

A linked list where navigation is possible both forward and backward.

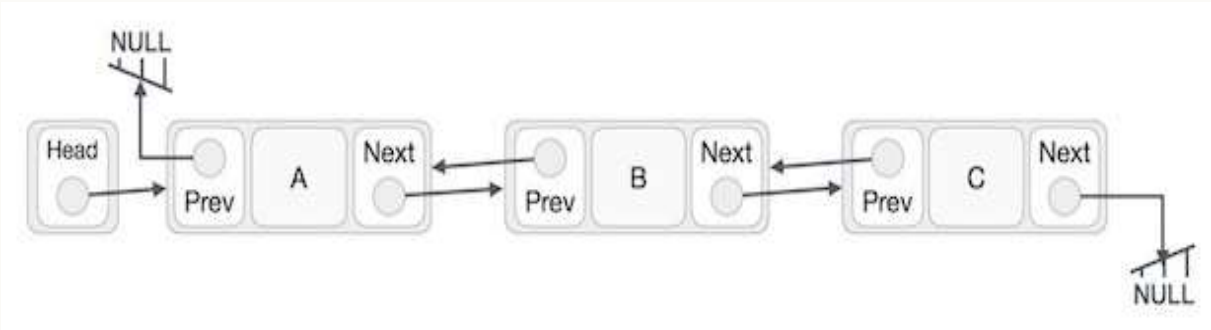
First & Last	The list tracks both a first and a last link.
next & prev	Each node has two links — next and previous.
Trade-off	Backward traversal is easy, but each node uses extra memory.

SECTION 03 · VARIATIONS

27

Doubly Linked List — Structure

Each node links both ways, so the list can be walked forward or backward.



SECTION 03 · VARIATIONS

28

Circular Linked List

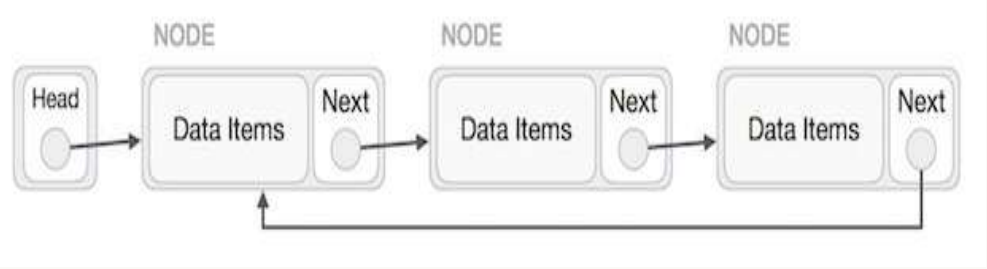
A linked list whose last node points back to the first — forming a loop.

- There is no NULL end — following next endlessly cycles through the list.
- Both singly and doubly linked lists can be made circular.
- Useful when you need to loop around the data continuously (e.g. round-robin scheduling).

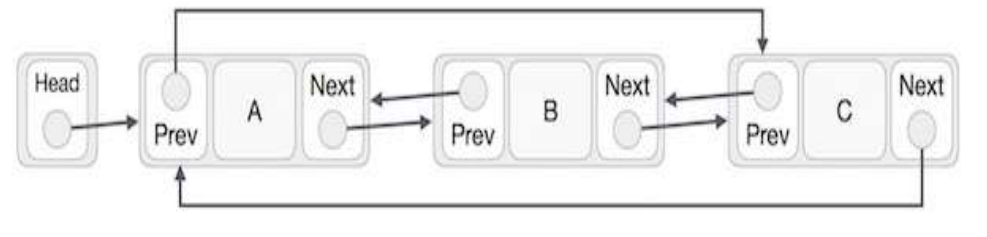
SECTION 03 · VARIATIONS

Circular Linked List — Structure

Singly Circular LL



Doubly Circular LL



SECTION 03 · VARIATIONS

How do you reverse a linked list?

An important question.

SECTION 03 · VARIATIONS

30

Reversing a Linked List

- Initialize three pointers: prev = NULL, curr = head, next = NULL.
- Iterate through the list. In each step:

Save the next node: `next = curr → next`

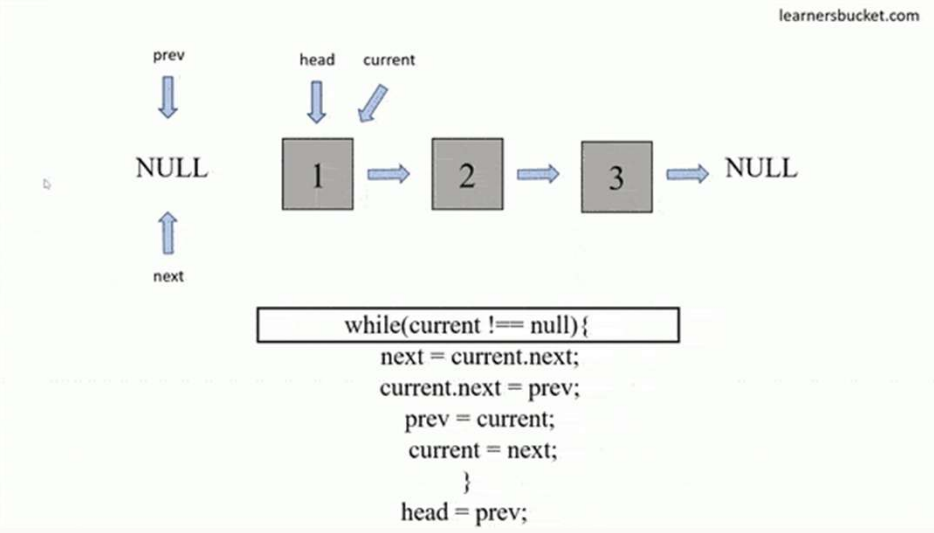
Flip the pointer: `curr → next = prev`

Advance both: `prev = curr,  curr = next`

SECTION 03 · VARIATIONS

31

Reversing a Linked List — Visual



SECTION 04

Trade-offs & Uses

Advantages, disadvantages, and where linked lists are used.

Advantages

- Dynamic size
- Efficient insertion & deletion
- Memory efficiency
- Easy to implement
- Flexibility
- Easy to navigate

SECTION 04 · TRADE-OFFS

33

Disadvantages

- Slow access time
- Higher memory overhead
- Extra memory per node
- Pointer overhead
- Cache inefficiency
- No random access

SECTION 04 · TRADE-OFFS

34

Applications

- Implementing stacks and queues.
- Representing trees and graphs.
- Dynamic memory allocation (linked list of free blocks).
- Representing sparse matrices and manipulating polynomials.
- Arithmetic on very long integers; finding paths in networks.
- Operating systems: memory management, process scheduling, file systems.

SECTION 04 · TRADE-OFFS

35

Sparse Matrix

A matrix in which most of the elements are zero.

- Storing every zero wastes huge amounts of memory for large matrices.
- Instead, store only the non-zero entries as (row, column, value) triples.
- A linked list of these triples is a natural, memory-efficient representation.

SECTION 04 · TRADE-OFFS

36

Sparse Matrix — Compact Storage

Keep only the non-zero cells — a big saving when most of the matrix is zero.

Sparse matrix: mostly zeros — store only the non-zero entries

Full 4×4 matrix (16 cells)

0	0	3	0
0	5	0	0
0	0	0	0
0	2	0	0

→

Store only 3 non-zero entries

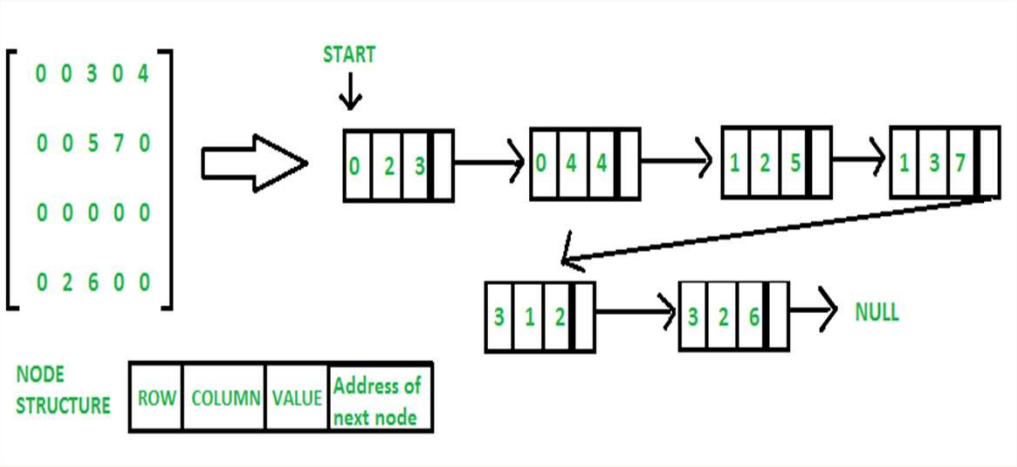
Row	Column	Value
0	2	3
1	1	5
3	1	2

Instead of 16 cells, store 3 (row, column, value) triples — a linked list of these saves memory for large sparse data.

SECTION 04 · TRADE-OFFS

37

Sparse Matrix — Compact Storage



SECTION 04 · TRADE-OFFS

38

Homework

Remove duplicates from a singly linked list

Given a singly linked list, remove all nodes that hold duplicate values (if any exist), keeping one occurrence of each value.

SECTION 05

Practice It Yourself

Linked-list problems are all about pointer manipulation — do them by hand, no AI.

PRACTICE · LINKED LISTS

P1

Problems to Try

Draw the nodes and pointers on paper first. That's how pointer intuition is built.

LeetCode 206	**Reverse Linked List**	Easy	Reverse a singly linked list; return the new head.
LeetCode 21	**Merge Two Sorted Lists**	Easy	Merge two sorted lists into one sorted list.
LeetCode 876	**Middle of the Linked List**	Easy	Find the middle node using slow & fast pointers.
LeetCode 141	**Linked List Cycle**	Easy	Detect whether the list loops, using two pointers.
LeetCode 234	**Palindrome Linked List**	Easy	Check if the list reads the same forwards and backwards.

PRACTICE · WORKED EXAMPLE

P2

LeetCode 206 — The Approach

Problem (in short): given the head of a singly linked list, reverse it and return the new head.

- Walk the list once, flipping each node's next pointer to point backwards.
- Keep three trackers: prev (starts NULL), curr (starts at head), next (temporary).
- Save next = curr.next so we don't lose the rest of the list.
- Flip it: curr.next = prev — this reverses one link.
- Advance: prev = curr, then curr = next — step forward.
- When curr becomes NULL, prev is the new head. Return prev.

YOUR TURN

On paper, reverse 1 → 2 → 3 → NULL. Write down prev, curr and next after each step. Where does the new head end up?

PRACTICE · WORKED EXAMPLE

P3

LeetCode 206 — The Code

```
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;

    while (curr != nullptr) {
        ListNode* next = curr->next;  // save the rest
        curr->next = prev;            // flip the link
        prev = curr;                  // advance prev
        curr = next;                  // advance curr
    }
    return prev;                      // new head
}
```

PRACTICE · WORKED EXAMPLE

P4

Why It Works & How Fast

The trick	Saving next before flipping curr.next is what stops us losing the rest of the list.
End state	The loop stops when curr is NULL; prev then points at the old last node — the new head.
Time	O(n) — each node is visited exactly once.
Space	O(1) — only three pointer variables, no extra list or recursion needed.

PRACTICE · RESOURCES

P5

Where to Get Help

Watch the idea only after you've tried it yourself.

NeetCode	Iterative + recursive walk-through. Search: "NeetCode Reverse Linked List 206"
Abdul Bari	Pointer-level explanations. Search: "Abdul Bari Linked List"
William Fiset	Full linked-list playlist for CS students. Search: "William Fiset Linked List"

YOUR TURN

Course rule: try each problem for at least 20 minutes and draw it out before watching anything — and never paste it into an AI.

LECTURE 2 — RECAP

What we covered

- Nodes, pointers, and the Head
- Linked list vs array trade-offs
- Insertion and deletion by re-linking
- Singly, doubly and circular lists
- Reversing a list with three pointers
- Applications: sparse matrices & more

Next: Recursion & Stacks